

Android 第六章第二节教学讲义（SQLite 数据库入门版）

各位同学，上节课咱们学了轻量级的数据存储方式 SharedPreferences，今天咱们来学习能存储大量结构化数据的 "大管家"——SQLite 数据库。比如手机通讯录、聊天记录这些需要频繁增删改查的数据，都适合用 SQLite 来管理。这节课咱们从 SQL 基础语法讲到 Android 中的具体用法，最后还用它优化 "记住密码" 功能，保证小白也能听懂！

一、先搞懂：SQL 基本语法（数据库操作的 "普通话"）

SQL（结构化查询语言）是操作数据库的通用语言，就像和数据库交流的 "普通话"。咱们先学最常用的几个语法：

1. 创建表（CREATE TABLE）

表是数据库中存数据的 "表格"，比如 "用户表"（user）包含 id、用户名、密码等列。

```
-- 创建用户表（user），包含3列：
-- id：整数，主键（唯一标识），自增长
-- username：文本，不能为null
-- password：文本，不能为null
CREATE TABLE user (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL,
  password TEXT NOT NULL
);
```

说明：AUTOINCREMENT表示 id 自动增长（不用手动设置），PRIMARY KEY表示主键（唯一不重复）。

2. 插入数据（INSERT）

往表中添加一行数据（一条记录）：

```
-- 往user表插入一条记录：用户名"admin"，密码"123456"  
INSERT INTO user (username, password) VALUES ("admin", "123456");
```

3. 查询数据（SELECT）

从表中读取数据：

```
-- 查询user表中所有记录（*表示所有列）  
SELECT * FROM user;  
-- 条件查询：查询用户名是"admin"的记录  
SELECT * FROM user WHERE username = "admin";
```

4. 更新数据（UPDATE）

修改表中的数据：

```
-- 将username为"admin"的密码改成"654321"  
UPDATE user SET password = "654321" WHERE username = "admin";
```

5. 删除数据（DELETE）

删除表中的数据：

```
-- 删除username为"admin"的记录  
DELETE FROM user WHERE username = "admin";
```

二、Android 中的 SQLite：用代码操作数据库

Android 内置了 SQLite 数据库，提供了 `SQLiteDatabase`（数据库操作类）和 `SQLiteOpenHelper`（数据库辅助类）来简化操作。

1. SQLiteOpenHelper：数据库的 "创建者和管理者"

它是一个抽象类，我们需要继承它，用来创建数据库、升级数据库。就像一个 "管家"，负责数据库的初始化和版本管理。

步骤 1：创建自定义的 Helper 类

```
public class MyDatabaseHelper extends SQLiteOpenHelper {
    // 数据库名
    private static final String DB_NAME = "user_db";
    // 数据库版本（必须是整数，升级时改大）
    private static final int DB_VERSION = 1;
    // 构造方法：上下文、数据库名、游标工厂（null即可）、版本号
    public MyDatabaseHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }
    // 数据库第一次创建时调用（初始化表）
    @Override
    public void onCreate(SQLiteDatabase db) {
        // 执行创建表的SQL语句
        String createTableSql = "CREATE TABLE user (" +
            "id INTEGER PRIMARY KEY AUTOINCREMENT," +
            "username TEXT NOT NULL," +
            "password TEXT NOT NULL)";
        db.execSQL(createTableSql);
    }
    // 数据库版本升级时调用（比如增加表或列）
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        // 后续讲版本升级时详细说
    }
}
```

2. SQLiteDatabase：数据库的 "操作工"

通过SQLiteOpenHelper获取SQLiteDatabase对象，用它执行增删改查操作。

获取 SQLiteDatabase 对象：

```
// 创建Helper实例
MyDatabaseHelper helper = new MyDatabaseHelper(this);
// 获取可写数据库（如果磁盘满了会报错）
SQLiteDatabase db = helper.getWritableDatabase();
// 也可以获取只读数据库（getReadableDatabase()）
```

三、实战：用 SQLite 实现增删改查（CRUD）

以 "用户表（user）" 为例，实现添加用户、查询用户、修改密码、删除用户的功能。

1. 增加数据（INSERT）

```
// 添加用户
public void addUser(String username, String password) {
    SQLiteDatabase db = helper.getWritableDatabase();
    // 方法1：用ContentValues（键值对，推荐，不用写SQL）
    ContentValues values = new ContentValues();
    values.put("username", username); // 列名：值
    values.put("password", password);
    // 插入数据：表名、nullColumnHack（一般为null）、values
    long rowId = db.insert("user", null, values);
    if (rowId != -1) {
        Toast.makeText(this, "添加成功", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "添加失败", Toast.LENGTH_SHORT).show();
    }
    db.close(); // 用完关闭数据库
}
```

2. 查询数据（SELECT）

```
// 根据用户名查询密码
public String queryPassword(String username) {
    SQLiteDatabase db = helper.getReadableDatabase();
    // 查询：表名、要查询的列（null表示所有列）、条件、条件值、分组、having、排序
    Cursor cursor = db.query(
```

```

        "user",
        new String[]{"password"}, // 只查password列
        "username = ?", // 条件: username=? (?是占位符, 防止SQL注入)
        new String[]{username}, // 条件值: 替换?
        null, null, null
    );
    String password = null;
    if (cursor.moveToFirst()) { // 判断是否有数据
        // 获取password列的值 (参数是列索引, 从0开始)
        password = cursor.getString(cursor.getColumnIndex("password"));
    }
    cursor.close(); // 关闭游标
    db.close(); // 关闭数据库
    return password;
}

```

3. 更新数据 (UPDATE)

```

// 修改密码
public void updatePassword(String username, String newPassword) {
    SQLiteDatabase db = helper.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put("password", newPassword); // 要修改的列和值
    // 更新: 表名、values、条件、条件值
    int rows = db.update(
        "user",
        values,
        "username = ?",
        new String[]{username}
    );
    if (rows > 0) {
        Toast.makeText(this, "密码修改成功", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "用户不存在", Toast.LENGTH_SHORT).show();
    }
    db.close();
}

```

4. 删除数据 (DELETE)

```
// 删除用户
public void deleteUser(String username) {
    SQLiteDatabase db = helper.getWritableDatabase();
    // 删除：表名、条件、条件值
    int rows = db.delete(
        "user",
        "username = ?",
        new String[]{username}
    );
    if (rows > 0) {
        Toast.makeText(this, "删除成功", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "用户不存在", Toast.LENGTH_SHORT).show();
    }
    db.close();
}
```

四、事务管理：保证操作的 "原子性"

事务就像 "打包操作"：要么所有操作都成功，要么都失败。比如转账时 "扣钱" 和 "加钱" 必须同时成功，不能只扣不加。

```
// 模拟转账：从A扣100，给B加100（用事务保证同时成功）
public void transferMoney() {
    SQLiteDatabase db = helper.getWritableDatabase();
    db.beginTransaction(); // 开启事务
    try {
        // 步骤1：A的余额减100
        ContentValues valuesA = new ContentValues();
        valuesA.put("balance", 900); // 假设A原来1000
        db.update("user", valuesA, "username = ?", new String[]{"A"});
        // 步骤2：B的余额加100
        ContentValues valuesB = new ContentValues();
        valuesB.put("balance", 1100); // 假设B原来1000
        db.update("user", valuesB, "username = ?", new String[]{"B"});
        db.setTransactionSuccessful(); // 标记事务成功
    } catch (Exception e) {
        db.rollback(); // 事务失败，回滚
    }
}
```

```
        Toast.makeText(this, "转账成功", Toast.LENGTH_SHORT).show();
    } catch (Exception e) {
        Toast.makeText(this, "转账失败", Toast.LENGTH_SHORT).show();
    } finally {
        db.endTransaction(); // 结束事务（如果标记成功则提交，否则回滚）
        db.close();
    }
}
```

五、数据库版本升级：给表 "添新功能"

当 APP 更新需要修改数据库（比如增加列），就需要升级版本：

1. 把DB_VERSION从 1 改成 2；
2. 在onUpgrade中处理升级逻辑：

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    // 如果旧版本是1，新版本是2，就添加"phone"列
    if (oldVersion < 2) {
        db.execSQL("ALTER TABLE user ADD COLUMN phone TEXT");
    }
    // 后续版本升级可以继续加if判断
}
```

六、优化 "记住密码" 案例：用 SQLite 替代 SharedPreferences

之前用 SharedPreferences 存账号密码，适合单用户；用 SQLite 可以存多个用户，更灵活。

登录界面逻辑优化：

```
public class LoginActivity extends AppCompatActivity {
    private EditText etUsername, etPassword;
    private CheckBox cbRemember;
```

```

private MyDatabaseHelper helper;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);
    helper = new MyDatabaseHelper(this);
    etUsername = findViewById(R.id.et_username);
    etPassword = findViewById(R.id.et_password);
    cbRemember = findViewById(R.id.cb_remember);
    findViewById(R.id.btn_login).setOnClickListener(v -> login());
}
private void login() {
    String username = etUsername.getText().toString().trim();
    String password = etPassword.getText().toString().trim();
    // 从数据库查询密码
    String savedPwd = queryPassword(username);
    if (savedPwd == null) {
        Toast.makeText(this, "用户不存在", Toast.LENGTH_SHORT).show();
        return;
    }
    if (!savedPwd.equals(password)) {
        Toast.makeText(this, "密码错误", Toast.LENGTH_SHORT).show();
        return;
    }
    // 登录成功：如果勾选记住密码，就更新数据库（如果是新用户则添加）
    if (cbRemember.isChecked()) {
        // 先查是否存在，不存在则添加
        if (savedPwd == null) {
            addUser(username, password);
        } else {
            // 已存在则更新（可选）
            updatePassword(username, password);
        }
    }
    Toast.makeText(this, "登录成功", Toast.LENGTH_SHORT).show();
}
// 调用前面写的增删改查方法...
private String queryPassword(String username) { ... }
private void addUser(String username, String password) { ... }
}

```

总结

这节课咱们学了 SQLite 数据库的核心知识：

1. **SQL 基本语法**：创建表（CREATE）、增（INSERT）、删（DELETE）、改（UPDATE）、查（SELECT）；
2. **SQLiteOpenHelper**：继承它来创建和管理数据库，重写 `onCreate`（初始化表）和 `onUpgrade`（版本升级）；
3. **SQLiteDatabase**：通过它执行增删改查，推荐用 `ContentValues` 避免手写 SQL；
4. **事务管理**：`beginTransaction()` 开启，`setTransactionSuccessful()` 标记成功，`endTransaction()` 结束，保证操作原子性；
5. **版本升级**：改大版本号，在 `onUpgrade` 中处理表结构修改；
6. **实战优化**：用 SQLite 替代 `SharedPreferences` 存账号密码，支持多用户。

SQLite 是 Android 开发的重要技能，建议课后做一个 "用户管理 APP"，实现注册（添加用户）、登录（查询用户）、修改密码、删除用户的功能，巩固今天的知识。有问题随时提问哦！

（注：文档部分内容可能由 AI 生成）